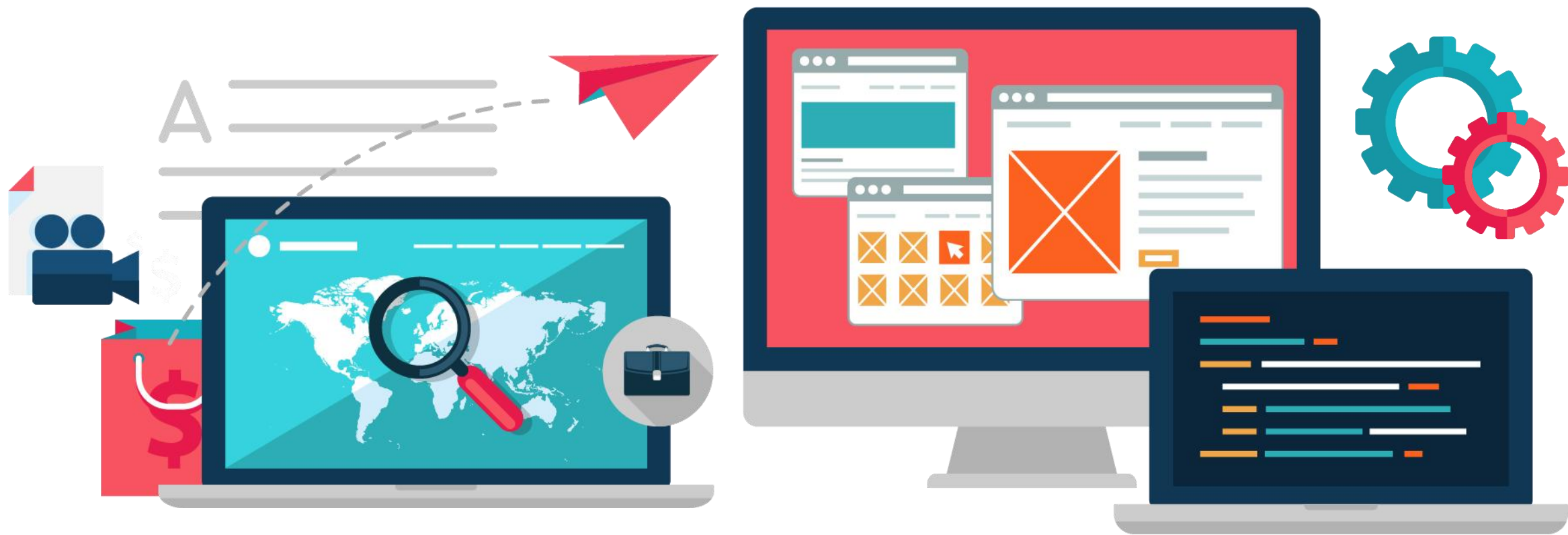


面向对象程序设计

Object Oriented Programming





类和对象



01

类型的抽象

02

类型的表示及定义

03

类定义

04

类的成员

05

类定义例

06

类和对象

07

对象的存储大小



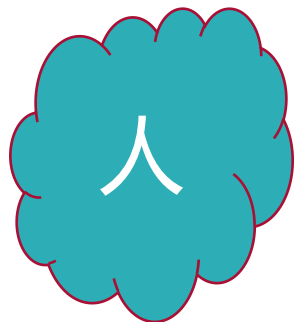
一个ADT：一个数学模型+可施加其上的操作集合。
(**类型名称**，**数据集**，**数据间的关系**，**操作集**)

- 与具体表示无关
- 与现实世界无关
- 任意性和无穷性

表示

```
a ADT :  
name {  
    数据:  
    关系:  
    操作:  
}
```

```
Person {  
    数据: age  
    关系: age >= 0  
    操作: birth  
           grow  
           dead  
}
```



```
Person {  
    数据: knowlege  
           fatigue  
    关系: knowlege > 0  
           fatgue >= 0  
    操作: study  
           sleep  
           play  
}
```



不同的关注，可导致不同的抽象结果

地球：-人口、经济等

行为：

增加人口

计算总人口

增加GDP

计算当年总GDP

计算总财富

数据：

人口数据

GDP数据

财富数据

地球：-太阳系中运动

行为：

指定年份的位置信息

与其他行星的位置关系

数据：

经纬度

赤道角

与太阳距离

线速度

...



书的抽象

行为:

返回书名
翻到第一页
翻到指定页
读N页
放置书签
添加批注
合上此书
...
放置到书架上
出版发行
传播信息
赠送
转卖
....

数据:

书名
总页数
当前页码
.....

行为的抽象

数据的抽象

水杯的抽象:

行为:

倒入N毫升水
被某人喝M毫升
倒掉

....

数据:

容量
水量

....



- 不同的关注，有不同的抽象
- 表示的方法多种
- 各计算机编程语言有各自的表示方法
- C++中用class表示类型的抽象

行为：

- 一个行为由一个或多个操作共同完成；
- 每个操作在C++通常用成员函数表示；

数据：

- 数据由数据成员表示

C++中用class表示，称类的定义。

- 类的定义(例)：

```
class ClassName {  
public:  
    void f ( ) ;  
    void g( ) ;  
private:  
    int  val;  
};
```

- 类定义的组成

- 数据成员 (或称成员数据)
- 成员函数 (或称函数成员)



C++的class是实现ADT的语法工具，当我们将一个ADT（如Stack）翻译成C++代码时，必须完成从“逻辑规范”到“物理实现”的跨越。

ADT 逻辑规范(做什么)	class 物理实现（做什么+如何做）
ADT名字	类名
操作集合(接口)	公有成员函数
预设存在数据集与状态	私有数据成员
操作规则	成员函数的实现逻辑



- ◆ 格式：`class <类名> {`
 ...
 };
- ◆ 定义顺序随意
- ◆ 通常放入头文件
- ◆ 可使用前置声明
- ◆ 类的构成
 - ▣ 数据成员 (或称成员数据)
 - ▣ 成员函数 (或称函数成员)
- ◆ *class与struct*

例：

```
//dog.h
#ifndef DOGH
#define DOGH

class Bone;

class Dog {
public:
    void setAge( int theAge);
    void eat( Bone & aBone);
protected:
private:
    int age;
};
#endif
```




类的定义-使用包含警戒 (反例说明)

反例:

```
//dog.h
#ifndef DOGH
#define DOGH

class Bone;

class Dog {
public:
    void setAge( int theAge);
    void eat( Bone & aBone);
protected:
private:
    int age;
};
#endif
```

反例:

```
//cat.h
#ifndef CATH
#define CATH

#include "dog.h"

...
#endif
```

```
//a.cpp
#include "dog.h"

...
```

```
//b.cpp
#include "dog.h"
#include "cat.h" //重复定义Dog

...
```



类的定义-可使用前置声明

```
//dog.h
#ifndef DOGH
#define DOGH
class Bone;
#include "cat.h" //递归包含
//使用class Cat; 解决互相依赖问题
class Dog {
public:
    void setAge( int theAge);
    void eat( Bone& aBone);
    Dog birth( );
    void fight(Cat& aCat);
private:
    int age;
};
#endif
```

作用:

■ 防止循环定义(递归包含)

结合包含警戒解决两个头文件互相包含、
两个类互相依赖的问题

```
//cat.h
#ifndef CATH
#define CATH
#include "dog.h" //递归包含
//使用class Dog; 可避免循环定义
class Cat {
public:
    void fight(Dog& aDog);
};
#endif
```



```
//dog.h
#ifndef DOGH
#define DOGH

class Bone; //对比#include "bone.h"
class Cat;

class Dog {
public:
    void setAge( int theAge);
    void eat( Bone & aBone);
    Dog birth( );
    void fight(Cat & aCat);
private:
    int age;
};
#endif
```

作用:

■ 降低文件间依赖性

- 1.节省编译时间，多余的#include会迫使编译器展开更多文件，处理更多输入
- 2.节省不必要的重新编译时间。#include使代码因为头文件中无关的改动而被重新编译多次。

```
//bone.h
#ifndef BONEH
#define BONEH

    ..修改之后..

#endif
```

```
//xxx.cpp
#include "dog.h

...
```



◆ 数据成员

- 普通类型（指针、数组、引用等）
- enum类型、const 类型、static
- struct 和class 不能递归定义，但可用指针

◆ 成员函数

```
class Dog {  
public:  
    Dog(Person& pers);  
    void run( );  
private:  
    Person & owner;  
    Leg    legs[4];  
    Ring * pRing;  
    int    age;  
    Dog    parter; //非法  
    Dog * parter; //合法  
};
```

例：(C++11)

```
enum class DogColor {  
    WHITE = 1,    BLACK = 2,    GRAY  = 3  
};
```

```
class Dog {  
    ...
```

```
private:
```

```
    const DogColor color = DogColor::WHITE;  
    const int age = 13;  
    float pi = 3.14;
```

//对静态数据成员，如果是整型或枚举类型
且被声明为const，那么可用常量表达式在类
体内直接初始化

```
    static const int v=50;
```

```
    static int k =60; //非法
```

```
    static const float vv =2.5; //非法
```

```
};
```

双向整数链表的行为

- ◆ 从后添加整数
- ◆ 从前添加整数
- ◆ 取得某个位置的整数
- ◆ 设置某个位置的整数
- ◆ ...

模拟赛车中的Car

- ◆ 启动
- ◆ 加速
- ◆ 减速
- ◆ 换挡位
- ◆ 刹车

- ◆ 撞车

```
class IntList {  
public:  
    void add(int n);  
    void preAdd(int n);  
    int  getInt(int pos) const;  
    void setInt(int pos,int n);  
    ...  
};
```

```
class Car {  
public:  
    void start( );  
    void speedUp();  
    void speedDown();  
    void shiftGear( int gear );  
    void brake( );  
    void hit( Car & otherCar);  
  
private:  
    ....  
};
```

分油问题：有三个油瓶子，容量分别为10斤、7斤、3斤。初始时，10斤瓶是满的，7斤和3斤瓶是空的。如何将油分成两个5斤？

```
//10斤倒7斤,(3,7,0)
//7斤倒3斤,(3,4,3)
//3斤倒10斤,(6,4,0)
//7斤倒3斤,(6,1,3)
//3斤倒10斤,(9,1,0)
//7斤倒3斤,(9,0,1)
//10斤倒7斤,(2,7,1)
//7斤倒3斤,(2,5,3)
//3斤倒10斤,(5,5,0)
```

如何用程序模拟验证这个过程呢？

```
class Bottle {
public:
    Bottle( int maxCapacity,int curUsed);

    void Pour(Bottle& other);

    int  GetUsed( ) const;

private:
    const int capacity; //最大容量

    int  used; //当前使用量

};
```



- 对象：类的实例化结果
 - 一个对象总是有类型的
 - 一个类型可实例化0个或多个对象
- 对象的定义(实例化)
 - 例： `[class] ClassName ObjectName;`
 - 例： `[class] ClassName * pObj = new ClassName;`

例：

1. 学生-张三
2. 学校-计算机学院
3. 菜单-清蒸鳊鱼
4. 书店-图书



- 对象的访问-调用函数与发送消息
 - 直接向对象发送消息
 - 向指针指向的对象发送消息
 - 向引用对象发送消息

面向对象中更强调发送消息，即发出消息，而接收对象及其相应行为是不确定的，也可能不是同步的。

```
// Account.h
#program once
class Account {
public:
    void add(int n);
    void transfer(Account& other,int n)
private:
    int amount = 0;
};
```

```
//main.cpp
#include "Account.h"
int main( ) {
    Account jack,tom;

    jack.add(20);
    tom.add(10);
    jack.transfer(tom,5);

    Account &ja = jack;
    ja.add(10);

    Account * pjack = &jack;
    pjack ->add(5);
}
```



```
//bottle.h  
  
#ifndef BOTTLEH  
#define BOTTLEH  
  
class Bottle {  
public:  
    Bottle( int maxCapacity,int curUsed);  
    void Pour(Bottle& other);  
    int GetUsed( ) const;  
private:  
    const int capacity; //最大容量  
    int used; //当前使用量  
};  
  
#endif
```

```
//main.cpp  
#include <iostream>  
#include "bottle.h"  
int main( ) {  
    Bottle b10(10,10),b7(7,0),b3(3,0);  
    b10.Pour(b7); //10斤倒7斤,(3,7,0)  
    b7.Pour(b3); //7斤倒3斤,(3,4,3)  
    b3.Pour(b10); //3斤倒10斤,(6,4,0)  
    b7.Pour(b3); //7斤倒3斤,(6,1,3)  
    b3.Pour(b10); //3斤倒10斤, (9,1,0)  
    b7.Pour(b3); //7斤倒3斤,(9,0,1)  
    b10.Pour(b7); //10斤倒7斤,(2,7,1)  
    b7.Pour(b3); //7斤倒3斤,(2,5,3)  
    b3.Pour(b10); //3斤倒10斤,(5,5,0)  
    std::cout<<"B10 = "<<b10.GetUsed()<<std::endl;  
    std::cout<<"B7 = "<<b7.GetUsed()<<std::endl;  
};
```

- ◆ 面向对象程序设计 = 类/对象 + 消息/关系，对照分油问题的代码
- ◆ 理解这个类是如何从问题中设计出来的，以及它体现了怎样的设计思想
- ◆ 第一步：问题分析——发现实体，聚焦行为
 - ✓ 按照面向对象的思维方式，首先观察问题域中存在哪些实体
分油问题中，实体非常明确：3个油瓶子，一个容量 10 斤的瓶子，初始是满的；一个容量 7 斤的瓶子，初始是空的；一个容量 3 斤的瓶子，初始是空的。这三个瓶子是真实存在于问题中的实体。每个瓶子都有固定的容量和可变的当前油量。
 - ✓ 面向对象设计更关心的不是它们含有什么属性，而是它们能做什么，即行为
一个瓶子在分油过程中能提供的行为：它可以把油倒给另一个瓶子，可以随时查询它里面还剩多少油。这就是从问题中抽象出来的行为。我们不关心倒油的算法怎么写，不关心油量用什么类型存，只关注两件事：倒油、查询。
 - ✓ 这种只关注对象能做什么、而暂不关心如何做的思维方式，就是行为的抽象

◆ 第二步：行为的抽象与接口契约(ADT定义)

- ✓ 行为的抽象需要一种清晰的表达方式，让所有相关方（设计者、实现者、调用者）都明白这些实体到底提供了什么行为，以及这些行为受什么规则约束。这种表达方式就是抽象数据类型，ADT是类型行为的逻辑规范和规格说明：

类型名称：瓶子（Bottle）

支持的行为：

1. 倒油：将当前瓶子中的油倒入另一个瓶子。
 - 倒油规则：一直倒，直到倒出瓶变空或倒入瓶变满为止。
 - 操作保证：一次倒油前后，参与倒油的两个瓶子的油量总和保持不变。
2. 查询油量：返回当前瓶内的油量。

必须遵守的约束：

- 每个瓶子的油量始终在0到其容量之间。
- ✓ 行为的抽象就是把实体对外提供的行为以及行为之间的逻辑关系抽取出来，形成一份独立的、与实现语言无关的契约

◆ 第三步：从行为抽象到C++类——兑现契约

✓ 行为的抽象（ADT）是蓝图，C++类是兑现蓝图的施工队

✓ 公有接口：行为抽象的直接映射

➤ `void Pour(Bottle& other);` 直接对应ADT中的倒油行为，参数是另一个瓶子的引用，表明倒油会直接修改两个对象的状态，与物理世界的交互方式一致。

➤ `int GetUsed() const;` 后缀强调查询行为不会修改对象状态，这是一个重要的行为约定。

➤ 公有接口直接表达类型的行为承诺。

✓ 私有成员：实现细节，与行为抽象无关

`private`部分是实现者选择的内部表示，行为抽象中从未规定容量必须用`const int`存，也从未规定油量必须叫`used`，可以改用其他类型或组织形式，只要`Pour`和`GetUsed`的外部行为保持不变，任何使用这个类的代码都感知不到内部表示的变化。

✓ 行为的抽象与数据的表示之间是分离的。行为的抽象是稳定的契约，数据的表示是可变的实现细节。设计类时，应该首先关注对象向外暴露的行为接口，而非其内部的数据表示。



◆ 第四步：使用验证

- ✓ 基于这个行为抽象写出的调用者代码极其清晰：
 - 调用者代码中没有出现任何倒油算法，算法等细节被封装在Pour的实现里
 - 调用者只依赖行为抽象中描述的倒油和查询两个行为，完全不触碰内部数据表示
 - 代码可读性极高，与自然语言描述的操作步骤几乎一一对应
- ✓ 这正是行为抽象带来的好处：调用者只需理解对象能做什么，无需关心对象如何做。

阶段	工作内容	核心要点
识别实体与行为	找到三个瓶子，抽象出倒油和查询两个行为	关注做什么，而非有什么
行为抽象(ADT)	描述行为及约束，不含实现细节	行为抽象是独立于语言的契约
定义C++类	将行为映射为公有接口，选择合适的数据表示	公有接口稳定，私有实现可变
验证	基于对象行为编写验证代码	依赖抽象接口，不依赖具体表示

- ◆ 面向对象设计的起点是发现实体和行为抽象，先明确对象能做什么，再考虑它有什么属性
- ◆ 类是ADT抽象规格的具体实现。抽象定契约，实现守契约，二者分离是软件健壮性的根基

◆ 与非静态数据成员的个数、类型有关

- 与成员函数的个数无关
- 与静态数据成员的多少、类型无关
- 与访问控制无关

◆ 一定非0

◆ 与是否含有虚函数有关

◆ 与编译时字节对齐方式设置有关

```
int main()
{
    A a1;
    sizeof(a1); sizeof(A);
};
```

```
class A {
public:
    void f( );
    int v1;                //4
private:
    int g( );
    int v2;                //4
    char c;                //1
    int va[2];              //4*2=8
    Person person;         //n
    Car * pCar;             //4
    Person& obj;            //4
    static int num;
};
```



◆ 结构成员对齐的原因

1.平台原因(移植原因): 一些资料上是这样说的, “不是所有的硬件平台都能访问任意地址上的任意数据; 某些硬件平台只能在某些特定地址处取某些特定的数据, 否则就会抛出硬件异常”。也就是说在读取内存数据时, 只能在规定的地址处读数据, 而不是任意地址的内存数据都是可以读取的。

2.效率原因: 正是由于只能在特定的地址处读取数据, 所以在访问一些数据时, 对于访问未对齐的内存, 处理器需要进行两次访问; 而对于对齐的内存, 只需要访问一次就可以。其实这是一种以空间换时间的做法, 但这种做法是值得的。



◆ 结构成员对齐的规则

1. 第一个成员在结构体变量偏移量为0的地址处，也就是第一个成员必须从头开始；
2. 其他成员变量要对齐到某个数字(对齐数)的整数倍的地址处。对齐数为编译器默认的对齐数与该成员大小中的较小值。vs中默认值是8，Linux默认值为4，可以通过`#pragma pack(n)`，但修改只能设置成1，2，4，8，16；
3. 结构体总大小为最大对齐数的整数倍，每个成员变量都有自己的对齐数；
4. 如果嵌套结构体，嵌套的结构体对齐到自己的最大对齐数的整数倍处，结构体的整体大小就是所有最大对齐数(包含嵌套结构体的对齐数)的整数倍。



本章结束